

phorce 튜토리얼

퀵 가이드로 로봇을 한 번 움직여 봤다면, 이제 **여러분의 코드**가 로봇을 보고 스스로 판단해 동작을 고르게 만듭니다. 쉬운 것부터 한 레슨씩 쌓아 갑니다. 파이썬을 먼저, 그다음 C++ 를 봅니다.

이 튜토리얼을 끝내면 할 수 있는 것

로봇의 관절 상태를 1 초에 1000 번 받아서, "이런 상황이면 → 이 동작을 재생"하는 여러분만의 규칙(또는 AI)을 코드로 만들 수 있습니다.

이 튜토리얼의 코드는 전부 실물 로봇에 붙습니다. `play()` 가 실행되면 로봇이 실제로 움직입니다 — 실행 전 항상 주변을 확인하세요. 코드 실수는 겁내지 않아도 됩니다: 이상한 요청은 로봇이 스스로 거절하고, 위급하면 물리 E-Stop 입니다.

레슨 0 준비 운동

SDK 예제들은 젯슨에 이미 설치돼 있습니다. 어디 있는지 먼저 찾아 둡니다.

```
# 예제 폴더 위치를 EX 라는 이름에 저장 (한 번만)
EX="$(ros2 pkg prefix phorce)/share/phorce/examples"
ls "$EX"
# 01_first_motion.py 02_read_feedback.py 03_feedback_to_motion.py raw_action.py
```

예제를 실행하는 법: `python3 "$EX/01_first_motion.py" 1` (뒤의 1 은 재생할 동작 번호).

레슨 1 로봇의 눈으로 보기 — 피드백 한 장

목표: 로봇이 보내는 관절 상태를 눈으로 확인한다.

먼저 코드 없이, 로봇이 뭘 보내는지 터미널에서 봅니다.

```
ros2 topic hz /phorce/feedback          # 1000 근처면 정상 (1 초에 1000 번)
```

```
ros2 topic echo /phorce/feedback --once # 딱 한 장을 펼쳐 보기
```

한 장 안에는 **관절 12 개** 각각의 정보가 들어 있습니다. 자주 쓰는 것만:

필드	뜻
<code>position_rad</code>	관절 각도 (라디안). 로봇이 지금 어느 각도인가
<code>velocity_rad_s</code>	관절이 도는 속도
<code>current_a</code>	모터에 흐르는 전류 (힘을 얼마나 쓰는가)
<code>temp_c</code>	모터 온도
<code>valid</code>	이 축 데이터를 믿어도 되는가 (아주 중요 — 아래 설명)

꼭 지킬 규칙 하나: 어떤 축 값을 쓰기 전에 그 축의 `valid` 가 참(**true**)인지 확인하세요. `valid` 가 참일 때만 그 숫자가 진짜입니다. "오래되지 않았으니 괜찮겠지"(!`stale`)로 대신하면 안 됩니다 — 아직 한 번도 안 온 축은 stale 도 아니거든요. "믿어도 된다"는 증거는 `valid` 하나뿐입니다.

레슨 2 파이썬으로 상태 읽기

목표: 코드로 로봇 상태를 받아서 화면에 뿌린다.

가장 쉬운 길은 `phorce` 라는 준비된 도구(파사드)를 쓰는 겁니다. 복잡한 ROS 설정을 대신 해 줍니다.

```
import phorce

# connect() 는 로봇(게이트웨이)에 붙는다. with 블록을 나가면 알아서 정리한다.
with phorce.connect() as robot:

    st = robot.status()          # 로봇 상태 요약 한 장 (읽기만 — 명령 안 보냄)

    print("지금 상태:", st.state_name)  # "IDLE" 이면 새 동작을 보내기 좋은 때
```

```
report = robot.doctor()          # 건강검진 (phorce doctor 명령과 같은 판정)
print("연결 정상?", report.ok)
```

실행: `python3 그파일.py`. 상태 이름이 출력되면 성공입니다. (`state_name` 이 알려주는 값들 — `IDLE` · `EXECUTING` · `RECOVERY_REQUIRED` 등 — 은 ④ API 레퍼런스의 Python 탭에 정리돼 있습니다.)

관절 값 하나하나를 직접 받고 싶다면 (rclpy)

더 자세한 1kHz 피드백을 직접 구독할 수도 있습니다. 이땐 **딱 한 가지 함정**이 있어요.

```
import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data    # ★ 이게 핵심
from agx_msgs.msg import PhorceFeedback

class Watcher(Node):
    def __init__(self):
        super().__init__("watcher")

        # ★ 반드시 qos_profile_sensor_data 로 구독할 것.
        #   그냥 구독하면 1kHz 발행자와 안 맞아서 "한 개도 안 옴" (조용히 침묵).
        self.create_subscription(
            PhorceFeedback, "/phorce/feedback", self.on_fb, qos_profile_sensor_data)

    def on_fb(self, msg):
        axis0 = msg.axis[0]

        if axis0.valid:          # ★ valid 먼저 확인
            print(f"0번 관절 각도: {axis0.position_rad:.3f} rad")

rclpy.init(); rclpy.spin(Watcher())
```

가장 흔한 실수: `qos_profile_sensor_data` 를 빼먹으면 에러도 안 나고 그냥 아무것도 안 옵니다. "왜 조용하지?" 싶으면 이걸 먼저 의심하세요.

레슨 3 핵심 — 보고 → 판단 → 움직이기

목표: 로봇 상태를 계속 보다가, 조건이 맞으면 동작을 재생한다. 이게 여러분 프로젝트의 뼈대입니다.

여기 **가장 중요한 규칙**이 있습니다. 두 가지 속도를 **섞지 마세요**.

빠른 일 (1 초 1000 번)

느린 일 (1 초 2 번쯤)

피드백을 받아서 **"최신 상태"만 저장**. 판단·전송은 절대 여기서 저장된 최신 상태를 보고 **판단**해서, 필요하면 **동작**
안 함 **재생**

```
import threading, time
import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
import phorce
from agx_msgs.msg import PhorceFeedback

latest = {"pos0": None} # 빠른 콜백이 최신값만 여기 저장

class Observer(Node): # 레슨 2 후반과 똑같은 1kHz 구독
    def __init__(self):
        super().__init__("observer")
        self.create_subscription(
            PhorceFeedback, "/phorce/feedback", self.on_fb, qos_profile_sensor_data)
    def on_fb(self, msg): # 1kHz 로 불림 — 저장만!
        a0 = msg.axis[0]
        if a0.valid:
            latest["pos0"] = a0.position_rad

rclpy.init()
threading.Thread(target=rclpy.spin, args=(Observer(),), daemon=True).start()
# ↑ 관측은 뒤에서 계속 돌게 둔다 — play() 가 기다리는 동안에도 멈추지 않게
```

```

with phorce.connect() as robot:
    while True:
        # 느린 판단 루프 (0.5 초마다)
        time.sleep(0.5)
        p = latest["pos0"]
        if p is None:
            continue
        # —— 여기가 여러분의 "머리" —— 규칙이든 AI 든 자유
        if p > 0.5:
            # 예: 0 번 관절이 많이 꺾였으면
            try:
                robot.play(2)
                # 2 번 동작 재생 (끝날 때까지 기다림)
            except phorce.MotionBusy:
                pass
                # 지금 바쁨 — 다음 번에 다시 (기다리면 풀림)
            except phorce.MotionRejected as e:
                print(e.detail)
                # 사람 조치 필요 (영점 버튼 등)
            break
            # 계속 돌아 봐야 거절만 쌓임 — 루프 중단

```

왜 1kHz 콜백에서 `play()` 를 부르면 안 되나요? 로봇은 한 번에 동작 하나만 받습니다(대기줄 없음). 1 초에 1000 번 `play` 를 부르면 거의 다 "바쁨(BUSY)"으로 버려집니다. 판단·전송은 느린 루프에서 하세요.

완성 예제: `python3 "$EX/03_feedback_to_motion.py"` 가 정확히 이 구조입니다. 이 파일을 복사해서 여러분 규칙만 바꾸면 프로젝트가 시작됩니다.

Ctrl+C 로 멈출 때 메시지가 요란해도 정상입니다. 위의 짧은 코드는 종료 처리를 생략해서, 끝낼 때 스레드

관련 경고가 몇 줄 쏟아질 수 있습니다 — 오류가 아닙니다. 깔끔하게 끝나는 종료 처리가 궁금하면 완성

예제 03 의 뒷부분을 보세요.

거절을 다루는 법

`play()` 는 상황에 따라 거절될 수 있습니다. 세 가지만 구분하면 됩니다.

예외	뜻	대응
----	---	----

`MotionBusy` 지금 다른 동작 중 기다리면 풀림. 다음 루프에 다시 시도

`MotionRejected` 준비 안 됨 / 복구 필요 등 사람이 개입. 영점 버튼 또는 복구 버튼

`MotionAborted` 재생 중 중단됨 메시지의 복구 절차를 따름

레슨 4 C++ 로도 (원한다면)

목표: 같은 일을 C++ 로. 실시간 성능이 필요할 때.

C++ 에서는 `phorce_cpp::motion_client` 를 씁니다. `CMakeLists.txt` 에:

```
find_package(phorce_cpp REQUIRED)
target_link_libraries(내프로그램 phorce_cpp::motion_client)

#include "phorce_cpp/motion_client.hpp"

auto client = phorce::MotionClient::attach(node, phorce::Target::robot());
auto op = client->play_async(7);      // 7번 동작 요청 (기다리지 않음)
auto fut = op->result();

// 타이머/루프 안에서 논블로킹으로 확인 (콜백 안에서 fut.get() 로 막지 말 것)
if (fut.wait_for(std::chrono::milliseconds(0)) == std::future_status::ready) {
    auto r = fut.get();
    if (r.ok()) { /* 성공 */ }
    else if (r.busy()) { /* 바쁨 — 나중에 다시 */ }
    else if (r.needs_operator()) { /* 사람이 버튼 눌러야 */ }
}
```

실행 예제: `ros2 run phorce_cpp phorce_example_01_first_motion 1 (_02_read_feedback, _03_feedback_to_motion`
도 있음).

재빌드하면 권한이 사라집니다. C++ 프로그램을 `colcon build` 로 다시 빌드하면 EtherCAT 권한(setcap)이 날아갑니다. 저수준 프로그램을 직접 돌린다면 `sudo agr-setcap-ethercat <실행파일>` 를 다시 실행하세요. (모션 슬롯 API 만 쓰면 해당 없음 — 권한은 게이트웨이가 가짐)

레슨 5 내 프로젝트로 확장하기

이제 재료가 다 모였습니다. 아이디어 몇 개:

- **규칙 기반:** "관절이 X 이상 꺾이면 인사 동작" 처럼 if 문으로 반응하는 로봇
- **센서 연동:** 여러분이 붙인 센서/카메라 값에 따라 다른 동작을 고르기
- **순서 재생:** 조건에 따라 여러 동작을 이어서 (단, 한 번에 하나씩, 이전 게 끝나면 다음)
- **AI 판단:** 피드백을 입력으로 받는 모델이 "몇 번 동작"을 출력하게

핵심은 항상 같습니다: **피드백을 보고(↑) → 판단 → 동작 번호를 보낸다(↓).**

막히면? 운영진에게 이 두 줄의 결과를 그대로 보여주세요:

`phorce doctor --json` 과 `phorce list --json`

phorce 해커톤 참가자 문서 ② 튜토리얼 — 함께 보기: ① 퀵 가이드 · ③ 매뉴얼(규칙·안전) · ④ **API 레퍼런스**(함수·필드 사전 — 코딩하다 막히면 여기) · ⑤ 시스템 안내